

Adding New Interface

Contents

Overview.....	2
Prerequisites	2
Step 1: Use script to create new interface directory.....	2
Step 2: Update header file for new interface	2
Step 3: Modify SConscript to build new channel.....	2
Step 4: Modify relevant ruby files	3
Step 5: Write the implementation file for your interface.....	3

Overview

Channels are the conduits used to communicate using the SimpleComms system. The SCS daemon process runs on each computer, and communicates with other SCS processes using CP/IP. User applications communicate with the local SCS daemon using UDP/IP. Most channels are duplex, but this doesn't have to be the case; a channel from digital compass, for example, could be simplex (output only).

Channels are designed to transmit a single interface class.

Prerequisites

Ensure that you have the CAT_DIR environment variable set to the root of the Pathfinder directory (the one containing config, doc, src).

Also, you should have reviewed the Pathfinder coding standards document before writing any code.

Step 1: Use script to create new interface directory

The first step is to use the available autogen perl script (generate.pl) to create a new interface directory. This includes the creation of a template header file, which will be updated in the next step.

e.g.

```
./prod/common/interfaces/autogen/generate.pl -i InterfaceName --csvable
```

Step 2: Update header file for new interface

A template header file was created in Step 1. This template includes the necessary serialize method.

NOTE:

- If the channel needs to be csv capable, make sure to...
 - add " : public csvable" after the class name
 - e.g.
class AisTestInterfaceStorage : public csvable
 - add "toCsv" function after the serialize function
 - e.g.
////////////////////////////////////
/// @brief Print structure out in CSV format
/// @param out - output stream
////////////////////////////////////
void toCsv(CsvOutputStream& out) const
{
 out("m_variable1",static_cast<int>(m_variable1)); //example of unsigned char, or unsigned_8 variable
 out("m_variable2", (m_variable2)); //example of double, bool, unsigned int, or std::string variable
}

Step 3: Modify SConscript to build new channel

The generate.pl script updates the interfaces/SConscript file to build the new interface. The new interfaces is added at the bottom of this file, but it should be in alphabetical order for easier maintenance. Make sure to edit this files so that the new interface is placed in the correct location, alphabetically.

Step 4: Modify relevant ruby files

Next, the interface's definition must be included in the global interface file, config/interfaces.rb. Probably the easiest way to modify this file is to copy and modify an existing channel definition. This file is a central location of all the channels within the system.

e.g.

```
InterfaceNameInput = createBasicInputInterface("InterfaceName")  
InterfaceNameOutput = createBasicOutputInterface("InterfaceName")
```

For a csvable channel to be included in a log, make sure to add the new interface to config/DataTypeTable.rb. There are two locations to update.

Step 5: Write the implementation file for your interface

The final step is to write the implementation for the newly defined class.

One good practice is to include get/set function calls to make retrieving and setting variables easier.

e.g.

```
// get/set variable1  
void set_variable1(const unsigned_8& m) { m_variable1 = m; };  
unsigned_8 get_variable1() const { return m_variable1; };
```